

Applied Machine Learning — Week 1

Introduction, ML Pipeline, Data, Evaluation Metrics

Comprehensive Revision Notes

Contents

1	What is Applied Machine Learning?	2
1.1	Scope of the Course	2
1.2	Example Applications	2
2	The ML Pipeline in Detail	2
3	Data Representation	3
3.1	The Data Frame	3
3.2	Data Types	3
4	Feature Preprocessing	3
4.1	Numerical Feature Preprocessing	3
4.2	Categorical Feature Preprocessing	4
5	Generalisation: Training vs. Test Error	4
5.1	The Central Goal	4
5.2	Overfitting: The Polynomial Example	5
5.3	The Complexity–Error Tradeoff	5
6	Train / Validation / Test Splits	5
6.1	Why Three Splits?	5
6.2	Cross-Validation	6
6.3	Validation for Non-IID Data	6
6.4	Common Pitfalls	6
7	Choosing a Model	6
7.1	No Free Lunch	6
7.2	How Much Data Do I Need?	7
8	Evaluation Metrics	7
8.1	Classification Metrics	7
8.1.1	Accuracy	7
8.1.2	Confusion Matrix and Derived Metrics	8
8.1.3	Thresholding Soft Classifiers	8
8.1.4	ROC Curve and AUC	8
8.1.5	Log-Loss (Binary Cross-Entropy)	9
8.2	Regression Metrics	9
9	How (Not) to Do Data Science	10

10 Key Definitions & Formulas Summary	10
11 Exam-Style Questions & Model Answers	11
12 Lecture Exercises & Worked Solutions	12
12.1 Exercise 1: Sensitivity to Scaling	12
12.2 Exercise 2: AUC and Class Imbalance	12
12.3 Exercise 3: Best Constant Predictor for MSE	13
12.4 Exercise 4: Best Constant Predictor for MAE	13

Fei Gao

What is Applied Machine Learning?

Scope of the Course

Applied ML is distinct from ML *theory* (asymptotic behaviour, PAC bounds, etc.). The focus is practical:

- How do we apply existing methods to new problems?
- How do we scale simple methods to large data?
- How is ML used in industry? When is ML *not* appropriate?
- What happens *after* a model is deployed in the real world?

Example Applications

Application	Type
Spam filtering	Binary classification
Product recommendations	Ranking / collaborative filtering
Credit-card fraud detection	Anomaly detection / classification
Motion capture	Sequence modelling
Character recognition & machine translation	Sequence-to-sequence
Voice recognition	Sequence classification
Identifying cancer subtypes	Clustering / unsupervised
Drug development	Regression / generative modelling

Connections:

Links to later topics Many of these applications reappear throughout the course: fraud detection uses the **class-imbalance** discussion (Section 8), recommendation systems use **matrix factorisation** (Week 8), cancer subtypes use **clustering** (Week 7).

The ML Pipeline in Detail

The 9-Step Pipeline

1. **Learn about the domain / application** — lots of work before any data. Identify a problem that *can* be solved with data. This is hard.
2. **Identify an appropriate task** — is this classification? Regression? Clustering? What is the target variable?
3. **Collect data** — in industry, this can be the majority of the work.
4. **Clean and preprocess data** — handle missing values, duplicates, outliers, encoding errors.
5. **Transform data or select useful features** — scaling, encoding, feature engineering, dimensionality reduction.
6. **Choose an appropriate method** — match the algorithm to the data and task.
7. **Train the model** — fit parameters to the training data.
8. **Evaluate, visualise, and interpret results** — use held-out data, appropriate metrics, visualisations.
9. **Deploy and monitor performance** — track for distribution shift, degradation over time.

Typically you cycle through these steps iteratively, refining at each pass.

Exam Alert:

In real-world / industry settings Steps 1–5 and 8–9 are often where the majority of effort goes. Step 7 (“fancy ML”) is a small fraction of the total work. The course mostly focuses on steps 5–8, but awareness of the full pipeline is essential.

Data Representation

The Data Frame

Most data comes in a **data frame**: rows are *instances* (objects, data points, samples) and columns are *attributes* (features, variables, fields).

The goal is typically to represent the data as a matrix $X \in \mathbb{R}^{N \times P}$, but getting there from raw data requires several preprocessing steps.

Data Types

Data Types in Practice

1. **Numerical** (real-valued) — continuous measurements (height, temperature, price).
2. **Categorical** — unordered discrete values (colour, country, species).
3. **Ordinal** — ordered discrete values (education level, satisfaction rating).
4. **Dates and times** — need special treatment (cyclical features, time-since-event).
5. **Coordinates** (geospatial) — latitude/longitude, distances, spatial features.
6. **Text** — bag-of-words, TF-IDF, embeddings.
7. **Unique identifiers** — IDs should *never* be used as features.
8. **Images, video, audio** — high-dimensional raw signals; typically need deep learning or specialised feature extraction.

Feature Preprocessing

Numerical Feature Preprocessing

Numerical Transforms

1. **Scaling and centering (whitening):**

$$x'_i = \frac{x_i - \bar{x}}{\sigma_x}.$$

Subtracts the mean and divides by the standard deviation. Many algorithms are sensitive to scale (e.g. k -NN, SVM with RBF kernel, gradient descent); tree-based methods are not.

2. **Rank transformation:** Replace each value with its rank. Makes the distribution uniform; robust to outliers.
3. **Clipping:** Cap extreme values at a fixed percentile (e.g. 1st and 99th).
4. **Nonlinear transformations:**
 - $\log(x) : \mathbb{R}^+ \rightarrow \mathbb{R}$ — useful for right-skewed data (incomes, counts).
 - $\text{logit}(x) : [0, 1] \rightarrow \mathbb{R}$ where $\text{logit}(x) = \log \frac{x}{1-x}$ — useful for proportions.
 - $\sqrt{x}$ — milder than log; useful for count data.

These can help with normality assumptions and make relationships more linear.

△ Sparse data

Centering sparse data (e.g. text bag-of-words) destroys sparsity — a sparse matrix becomes dense, exploding memory usage. Consider scaling without centering, or use algorithms that handle sparse data natively.

Exam Alert:

Which models are sensitive to scaling?

Sensitive to scaling	Invariant to scaling
k -NN (distance-based)	Decision trees (threshold-based)
SVM (especially RBF kernel)	Random forests
Linear regression (if comparing coefficients)	Gradient boosting (tree-based)
Neural networks	Naïve Bayes
PCA (maximises variance)	

Categorical Feature Preprocessing

Categorical Encoding Methods

- One-hot (dummy variable) encoding:** Create a binary column for each category value. E.g. colour $\in \{\text{red, green, blue}\} \rightarrow (1, 0, 0), (0, 1, 0), (0, 0, 1)$.
 - Most common and safest default.
 - Problem: high cardinality $\Rightarrow$ very high-dimensional, sparse features.
- Label (ordinal) encoding:** Map categories to integers: red $\rightarrow$ 0, green $\rightarrow$ 1, blue $\rightarrow$ 2.
 - Only appropriate if there is a genuine natural ordering!
 - Otherwise imposes a false ordinal relationship.
- Frequency (count) encoding:** Replace each category with its frequency in the dataset.
 - Example: bag-of-words for text data.
 - Loses the identity of rare categories (all singletons map to 1).
- Target (mean) encoding:** Replace each category with the mean of the target variable for that category.
 - Pitfall: **data leakage** if not done carefully (target info leaks into features).
 - Must use cross-validated or Bayesian smoothed versions.
- Learned embeddings:** Map each discrete value onto a vector in $\mathbb{R}^D$, trained end-to-end.
 - Common in NLP (word/character embeddings) and recommender systems.
 - Requires a neural network architecture.

Generalisation: Training vs. Test Error

The Central Goal

The Central Goal of ML

In machine learning, we care about the error on **test data** — data the model has never seen during training.

Analogy: Training error is your score on a practice exam. Test error is your score on the real exam. Memorising the practice exam gives low training error but does *not* guarantee low test error. **Learning** (generalisation) is necessary.

Overfitting: The Polynomial Example

Setup: 8 data points, fit a polynomial $f(x) = \sum_{i=1}^M \beta_i x^i$ with M parameters.

- $M = 0$ (constant): extreme underfitting, high training error.
- $M = 1$ (line): simple model, reasonable fit.
- $M = 3-5$: good balance, captures the trend.
- $M = 7$: **perfect interpolation** (8 points, 8 parameters) — zero training error but the curve oscillates wildly between data points. Terrible generalisation.

Lesson: more parameters $\neq$ better. When parameters $\geq$ data points, the model can memorise (interpolate) the training data exactly, but this is overfitting.

The Complexity–Error Tradeoff

Training vs. Generalisation Curves

As model complexity increases:

- **Training accuracy** monotonically increases (eventually reaching 100%).
- **Generalisation (test) accuracy** first increases, then decreases.

The gap between training and test accuracy grows with complexity — this is the hallmark of overfitting.

The optimal complexity is where **test accuracy is maximised** (the “sweet spot”).

Fundamental Assumption for Generalisation

Test data must be similar to training data. If training and test data are unrelated, we cannot learn anything useful. Generalisation requires assumptions about the data-generating process.

Train / Validation / Test Splits

Why Three Splits?

The Three-Way Split

1. **Training set:** used to fit model parameters.
2. **Validation set:** used to tune hyperparameters (model selection, regularisation strength, architecture choices). Provides an unbiased estimate of performance during development.
3. **Test set:** used *once* at the very end to estimate final performance. Never used during training or model selection.

△ Don't overfit to the test set!

If you repeatedly evaluate on the test set and make decisions based on those results, you are effectively using the test set for model selection — and your reported test performance is no longer an unbiased estimate.

Cross-Validation

K -Fold Cross-Validation

1. Hold out the test set entirely.
 2. Split the remaining training data into K folds.
 3. For each fold $k = 1, \dots, K$: train on $K-1$ folds, validate on fold k .
 4. Average the K validation scores to estimate generalisation performance.
- Common choices: $K = 5$ or $K = 10$. Extreme case: $K = N$ is **leave-one-out** (LOO) CV. Cross-validation is used to select hyperparameters and compare models. After selection, retrain on the full training set and evaluate on the held-out test set.

Validation for Non-IID Data

Non-IID Data Splits

Data is often *not* independent and identically distributed! A random split might be inappropriate.

Grouped data (e.g. data from different cities, classrooms, patients):

- Within-group correlation is higher than between-group.
- Use **stratified sampling**: ensure each fold has a representative distribution of groups.
- Or use **group-level splits**: entire groups go into train or validation, never split across.

Time-series data:

- Random splits test *interpolation* between time points.
- In deployment, you need to *extrapolate* into the future.
- Use **temporal splits**: validation/test data is always *after* training data chronologically.
- Careful: more data than you think is time-series data! Data collected sequentially over months/years.

Common Pitfalls

△ Don't look at the test set!

It is easy to accidentally leak information from the test set:

- **Computing scaling transforms**: if you compute mean/std on the entire dataset (including test), the training data “sees” test statistics. Always fit transforms on training data only, then apply to test.
- **Exploratory data analysis**: visualising the full dataset (including test) can unconsciously influence modelling choices.
- **Feature selection**: selecting features based on correlation with labels across the full dataset leaks test information.

Choosing a Model

No Free Lunch

There is no single “best” ML algorithm. The right choice depends on:

- Data size (N) and dimensionality (D).
- Data types (numerical, categorical, mixed, text, images).
- Whether interpretability is required.
- Computational budget (training and inference time).
- Whether uncertainty estimates are needed.

Connections:

Links to later topics The course progressively builds a toolbox: **linear models** (Week 2), **kernel methods** (Week 2), **trees** (Week 3), **ensembles** (Week 3), **unsupervised methods** (Weeks 7–8). The “off-the-shelf” comparison table from the ESL book (Week 3 boosting slides) rates methods across robustness, scalability, interpretability, and predictive power.

How Much Data Do I Need?

Two Types of Data Sources

Cheap (infinite) data	Expensive data
Predict observable events:	Automate complex processes:
Ad clicks	Medical diagnosis
Product recommendations	Drug trials
Stock market	Microchip design
House numbers	
Transcription/autocorrect	

Big Data Headaches

More data is *not always* better:

1. **Diminishing returns vs. engineering cost:** can you get away with a subset that fits in RAM? Data scientist time is expensive.
2. **Representativeness:** where did the data come from? Is it representative of future data? (Distribution shift.)
3. **Quality vs. quantity:** would a smaller dataset of higher quality be more useful? (Depends on the problem — sometimes yes, sometimes no.)

Evaluation Metrics

Classification Metrics

Accuracy

Accuracy

$$\text{Accuracy} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}[f(x_i) = y_i] = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}.$$

Limitations:

1. Uses **hard predictions** — does not take into account uncertainty or confidence.
2. **Non-differentiable** — cannot be optimised directly with gradient-based methods (use surrogate losses instead).
3. **Misleading for imbalanced classes** — e.g. 99% class 0: always predicting 0 gives 99% accuracy but zero recall on the minority class.

Confusion Matrix and Derived Metrics

The Confusion Matrix

For binary classification with **positive** and **negative** classes:

		Predicted	
		Positive	Negative
Actual	Positive	TP	FN
	Negative	FP	TN

From this we derive:

- **Precision:** “Of those *predicted* positive, how many are actually positive?”

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}.$$

- **Recall (Sensitivity, True Positive Rate):** “Of those *actually* positive, how many did we correctly predict?”

$$\text{Recall} = \text{TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}}.$$

- **Specificity (True Negative Rate):** “Of those *actually* negative, how many did we correctly identify?”

$$\text{Specificity} = \text{TNR} = \frac{\text{TN}}{\text{TN} + \text{FP}}.$$

- **False Positive Rate:** $\text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}} = 1 - \text{Specificity}.$

Thresholding Soft Classifiers

Threshold Selection

Suppose $f(x_i)$ returns a real value (score or probability). We define:

$$\hat{y}_i = \begin{cases} 1 & \text{if } f(x_i) \geq c, \\ 0 & \text{if } f(x_i) < c, \end{cases}$$

where c is the threshold. Different values of c yield different trade-offs:

- **Low threshold:** more positives predicted $\Rightarrow$ high recall, low precision (many false positives).
- **High threshold:** fewer positives predicted $\Rightarrow$ high precision, low recall (many false negatives).

ROC Curve and AUC

AUC-ROC

The **ROC curve** plots TPR (recall) vs. FPR across all thresholds c .

- **Perfect classifier:** point $(0, 1)$ — 100% TPR, 0% FPR.
- **Random classifier:** diagonal line from $(0, 0)$ to $(1, 1)$.
- **Curves above the diagonal:** better than random.

The **Area Under the Curve (AUC)** summarises performance across all thresholds:

- $\text{AUC} = 1.0$: perfect.

- AUC = 0.5: random.
- AUC < 0.5: worse than random (flip your predictions!).

Advantage: threshold-independent — evaluates the *ranking quality* of the classifier.

Note: consider how AUC is affected by class imbalance. With extreme imbalance, Precision–Recall curves can be more informative.

Log-Loss (Binary Cross-Entropy)

Log-Loss

If we interpret the model output $f_\theta(x_i) \in [0, 1]$ as the probability $p(y_i = 1 \mid x_i; \theta)$, then the log-likelihood is:

$$\log p(\mathbf{y} \mid X; \theta) = \sum_{i=1}^N [y_i \log f_\theta(x_i) + (1 - y_i) \log(1 - f_\theta(x_i))].$$

The **log-loss** (binary cross-entropy) is the negation:

$$\ell(\theta) = - \sum_{i=1}^N [y_i \log f_\theta(x_i) + (1 - y_i) \log(1 - f_\theta(x_i))].$$

This is the standard loss for logistic regression and neural-network classifiers. It is:

- **Differentiable** — can be optimised with gradient descent.
- **Probabilistic** — directly measures how well the predicted probabilities match the true labels.
- **Proper scoring rule** — minimised when predicted probabilities equal true probabilities.

Maximising the log-likelihood (MLE) is equivalent to minimising the log-loss.

Regression Metrics

Regression Metrics

Mean Squared Error (MSE):

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (f(x_i) - y_i)^2.$$

- Easy to optimise (differentiable, second-order methods applicable).
- $\text{RMSE} = \sqrt{\text{MSE}}$ (same units as y).
- Analogous to assuming normally distributed errors: $y_i \sim \mathcal{N}(f(x_i), \sigma^2)$.
- **Best constant predictor** = $\bar{y}$ (the mean). *Proof:* differentiate $\sum (y_i - c)^2$ w.r.t. c , set to zero.
- Sensitive to outliers (large errors get squared).

Mean Absolute Error (MAE):

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |f(x_i) - y_i|.$$

- Also easy to optimise (but not by second-order methods — not differentiable at zero).
- **Less sensitive to outliers** than MSE.

- Analogous to assuming Laplace-distributed errors.
- **Best constant predictor** = $\text{median}(y)$.

Connections:

MLE connection MSE corresponds to maximum likelihood under Gaussian noise. MAE corresponds to maximum likelihood under Laplace noise. This connects to the **Ridge regression** $\leftrightarrow$ **Gaussian prior** and **LASSO** $\leftrightarrow$ **Laplace prior** correspondence in Week 2.

How (Not) to Do Data Science

The lecture concludes with four **anti-patterns** — things you should *never* do:

Exam Alert: Common Bad Practices

1. “There’s no need to look at the data before running a model (the AI will handle it).”
Always perform exploratory data analysis (EDA) first.
2. “Pick a model based on hype or social-media mentions.”
Choose based on the problem structure, data type, and scale.
3. “Don’t worry about downstream tasks, users, or what happens after deployment — just maximise accuracy.”
Consider fairness, interpretability, robustness, and deployment constraints.
4. “Don’t bother comparing performance against simple baselines.”
Always start with a baseline (e.g. majority class, mean predictor, linear model). If your complex model doesn’t beat the baseline, something is wrong.

Key Definitions & Formulas Summary

Concept	Formula / Definition
Accuracy	$\frac{1}{N} \sum \mathbb{I}[f(x_i) = y_i]$
Precision	$\text{TP}/(\text{TP} + \text{FP})$
Recall (Sensitivity)	$\text{TP}/(\text{TP} + \text{FN})$
Specificity	$\text{TN}/(\text{TN} + \text{FP})$
FPR	$\text{FP}/(\text{FP} + \text{TN}) = 1 - \text{Specificity}$
AUC-ROC	Area under TPR vs FPR curve
Log-loss (BCE)	$-\sum [y_i \log f_\theta(x_i) + (1 - y_i) \log(1 - f_\theta(x_i))]$
MSE	$\frac{1}{N} \sum (f(x_i) - y_i)^2$; best constant = $\bar{y}$
MAE	$\frac{1}{N} \sum f(x_i) - y_i $; best constant = $\text{median}(y)$
Logit transform	$\log \frac{x}{1-x} : [0, 1] \rightarrow \mathbb{R}$
Standardisation	$(x - \bar{x})/\sigma_x$

Exam-Style Questions & Model Answers

Exam Q&A:

Q1 — Class Imbalance (Conceptual, 2 marks) **Q:** A dataset has 10,000 examples: 9,900 are class 0 and 100 are class 1. A classifier always predicts class 0. (a) What is its accuracy? (b) What is its recall on class 1? (c) What metric would you use instead?

A: (a) Accuracy = $9900/10000 = 99\%$. (b) Recall on class 1 = $0/100 = 0\%$ — the classifier completely fails on the minority class. (c) Use precision–recall curve, F1-score, or AUC-ROC, which account for performance on both classes.

Exam Q&A:

Q2 — Cross-Validation (Short-answer, 3 marks) **Q:** Explain why you should not compute feature scaling statistics on the full dataset before splitting into train/validation/test.

A: Computing the mean and standard deviation on the full dataset (including test data) leaks information from the test set into the training process. The model indirectly “sees” test data through the scaling parameters. This produces overly optimistic performance estimates. Correct procedure: compute statistics on the training fold only, then apply those statistics to transform validation and test data.

Exam Q&A:

Q3 — Overfitting (Short-answer, 3 marks) **Q:** You fit a degree-7 polynomial to 8 data points and achieve zero training error. (a) Should you expect good test performance? (b) What would you change?

A: (a) No. With 8 parameters and 8 data points, the polynomial interpolates exactly (zero training error) but oscillates wildly between data points, producing poor generalisation. (b) Reduce model complexity (lower degree), use regularisation (e.g. ridge penalty on coefficients), or collect more data. Cross-validation on degree M would identify the optimal complexity.

Exam Q&A:

Q4 — Metrics (Formula + Comparison, 3 marks) **Q:** Write the MSE and MAE formulas. Which is more robust to outliers and why? What are the best constant predictors for each?

A: $\text{MSE} = \frac{1}{N} \sum (f(x_i) - y_i)^2$, $\text{MAE} = \frac{1}{N} \sum |f(x_i) - y_i|$. MAE is more robust because it penalises errors linearly rather than quadratically — a single large outlier has a bounded effect on MAE but can dominate MSE. Best constant for MSE is the mean $\bar{y}$ (minimises sum of squared deviations). Best constant for MAE is the median (minimises sum of absolute deviations).

Exam Q&A:

Q5 — Time Series (Conceptual, 2 marks) **Q:** Why is random K -fold CV inappropriate for time-series forecasting?

A: Random splits allow future data into the training set and past data into the validation set, testing *interpolation* rather than *extrapolation*. In deployment, the model must predict future values from past data. Temporal splits (validation always after training chronologically) simulate this realistic scenario.

Exam Q&A:

Q6 — Log-Loss (Derivation, 3 marks) **Q:** Derive the binary cross-entropy loss from the maximum likelihood principle for a Bernoulli model.

A: For $y_i \in \{0, 1\}$ and predicted probability $p_i = f_\theta(x_i)$, the Bernoulli likelihood is $p(y_i | x_i; \theta) = p_i^{y_i} (1 - p_i)^{1 - y_i}$. Assuming independence:

$$p(\mathbf{y} | X; \theta) = \prod_i p_i^{y_i} (1 - p_i)^{1 - y_i}.$$

Log-likelihood: $\sum_i [y_i \log p_i + (1 - y_i) \log(1 - p_i)]$. MLE maximises this $\Leftrightarrow$ minimises the negation, which is the binary cross-entropy loss.

Lecture Exercises & Worked Solutions

These are the exercises posed in the lecture slides, collected here with full worked solutions.

Exercise 1: Sensitivity to Scaling

Exam Alert: From Slide 21

Question: What machine learning models are sensitive to scaling? What ones are not?

Solution:

Sensitive to feature scaling:

- **k-Nearest Neighbours:** uses Euclidean (or other) distance. A feature measured in thousands (e.g. salary in pounds) dominates one measured in single digits (e.g. number of rooms). Without scaling, the distance is effectively computed in only the high-magnitude dimension.
- **SVM (especially with RBF kernel):** the RBF kernel $\kappa(x, z) = \exp(-\|x - z\|^2 / 2\sigma^2)$ depends directly on Euclidean distance, so unscaled features with large magnitudes dominate.
- **Linear / logistic regression** (when comparing coefficients): the magnitude of w_d depends on the scale of feature d . Scaling is needed if you want to interpret $|w_d|$ as “importance.” For prediction alone, the model compensates via the coefficients, but **gradient descent convergence** is slower when features are on very different scales (elongated loss contours).
- **Neural networks:** same gradient-convergence issue as linear models, amplified by depth.
- **PCA:** maximises variance, so a feature in large units contributes disproportionately.
- **LASSO / Ridge:** the regularisation term $\lambda \sum |w_d|$ or $\lambda \sum w_d^2$ treats all coefficients equally, so a feature on a larger scale gets penalised differently than an equivalent feature on a smaller scale.

Invariant to monotone scaling:

- **Decision trees / CART:** splits are based on thresholds ($x_d \leq c$). Any monotone rescaling of a feature simply changes the threshold value, not the split structure.
- **Random forests / Gradient boosting:** composed of decision trees, so inherit their scale invariance.
- **Naïve Bayes:** models $p(x_d | y)$ per feature. The distribution parameters adjust to the scale.

Exercise 2: AUC and Class Imbalance

Exam Alert: From Slide 62

Question: Think about how AUC-ROC is affected by class imbalance.

Solution:

AUC-ROC measures the trade-off between TPR and FPR. Under **severe class imbalance** (e.g. 1% positive, 99% negative), there are two key effects:

1. **FPR can be very low even with many false positives in absolute terms.** If the negative class has 9,900 examples, even 100 false positives gives $\text{FPR} = 100/9900 \approx 1\%$. The ROC curve can look excellent (high AUC) even though 100 of your “positive” predictions are wrong — i.e. precision is low.
2. **A high AUC does not guarantee useful predictions.** You could have $\text{AUC} = 0.95$ but, at any reasonable operating point, the number of false positives far exceeds the true positives, making the classifier impractical (e.g. for medical screening or fraud detection).

Better alternative for imbalanced data: the **Precision–Recall (PR) curve** and **Average Precision (AP)** metric. These focus on performance among predicted positives, which is what typically matters when the positive class is rare.

Intuition: ROC-AUC measures how well you rank positives above negatives overall. PR-AUC measures how well you concentrate positives at the top of your ranking, which is more demanding when positives are rare.

Exercise 3: Best Constant Predictor for MSE

Exam Alert: From Slide 64

Question: For the mean squared error, what is the best constant predictor?

Solution:

We seek $c^* = \arg \min_c \frac{1}{N} \sum_{i=1}^N (y_i - c)^2$.

Derivation:

$$\frac{\partial}{\partial c} \sum_{i=1}^N (y_i - c)^2 = \sum_{i=1}^N 2(c - y_i) = 2Nc - 2 \sum_{i=1}^N y_i.$$

Setting to zero:

$$2Nc - 2 \sum_{i=1}^N y_i = 0 \implies c^* = \frac{1}{N} \sum_{i=1}^N y_i = \bar{y}.$$

The second derivative is $2N > 0$, confirming this is a minimum.

Interpretation: The mean minimises the sum of squared deviations. This is a fundamental property of the mean and connects to the fact that MSE corresponds to a Gaussian likelihood (the MLE of the mean of a Gaussian is the sample mean).

Exercise 4: Best Constant Predictor for MAE

Exam Alert: From Slide 65

Question: For the mean absolute error, what is the best constant predictor?

Solution:

We seek $c^* = \arg \min_c \frac{1}{N} \sum_{i=1}^N |y_i - c|$.

Derivation:

The absolute value function is not differentiable at zero, so we use a sub-gradient argument. The derivative of $|y_i - c|$ w.r.t. c is:

$$\frac{\partial}{\partial c} |y_i - c| = \begin{cases} -1 & \text{if } c < y_i, \\ +1 & \text{if } c > y_i. \end{cases}$$

Summing over all i :

$$\frac{\partial}{\partial c} \sum_{i=1}^N |y_i - c| = |\{i : c > y_i\}| - |\{i : c < y_i\}|.$$

This equals zero when the number of points above c equals the number below c , i.e. when c is the **median** of $\{y_1, \dots, y_N\}$:

$$c^* = \text{median}(y_1, \dots, y_N).$$

Interpretation: The median minimises the sum of absolute deviations. MAE corresponds to a Laplace likelihood (the MLE of the location parameter of a Laplace distribution is the sample median). This is why MAE is more robust to outliers than MSE: the median is much less sensitive to extreme values than the mean.

Comparison with Exercise 3:

	MSE	MAE
Best constant predictor	Mean $\bar{y}$	Median of y
Corresponding distribution	Gaussian	Laplace
Sensitivity to outliers	High (squared penalty)	Low (linear penalty)
Differentiable?	Yes	No (at $y_i = c$)